build_Krause_text

2026 年 3 月 30 日

1 Local Coupling + Top-k + RBF Kernel

基于 Krause 模型的文本注意力机制：每个 token 在局部窗口内寻找语义最相近的 k 个邻居，通过 RBF 核加权聚合信息。

1.1 简化部分：

- Multi-head -> Single-head
- Batch_size = 1

1.2 理论公式

对于第 i 个 token，其更新规则为：

$$\mathbf{x}_i^{(t+1)} = \text{LayerNorm}\left(\mathbf{x}_i^{(t)} + \sum_{j \in \mathcal{N}_k(i)} w_{ij} \cdot \mathbf{x}_j^{(t)}\right)$$

其中：

- $\mathcal{N}_k(i)$：在局部窗口内距离最近的 k 个邻居
- w_{ij}：由 RBF 核计算的注意力权重

$$w_{ij} = \frac{\exp(-\gamma \cdot d_{ij}^2)}{\sum_{l \in \mathcal{N}_k(i)} \exp(-\gamma \cdot d_{il}^2)}$$

- $d_{ij} = \|\mathbf{x}_i - \mathbf{x}_j\|_2$：L2 距离
- $\gamma = \frac{1}{2\sigma^2}$：可学习的核参数，控制距离衰减速度

1.3 代码实现

[38]:
```
import torch
import torch.nn as nn
import torch.nn.functional as F

class KrauseTextAttention(nn.Module):
    def __init__(self, d:int, k:int, window:int, gamma:float):
        super().__init__()
        assert k <= 2 * window, f"k={k} 不能超过窗口内邻居数量 {2*window}"
        self.k = k
        self.gamma = nn.Parameter(torch.tensor([gamma])) # \gamma = \frac{1}{2␣
↪\sigma ^2} 为可学习的
        self.window = window
        self.d = d
        self.layernorm = nn.LayerNorm(d)
        # 保存中间结果用于调试
        self._attention_weights = None
        self._topk_dist = None

    def forward(self, X:torch.Tensor):
        # N = Token 个数
        # d = Embedding Space 维度
        # w = 窗口半径
        N,_ = X.shape
        w = self.window

        #  关键修复: L2 归一化, 让距离落在合理范围
        X_norm = F.normalize(X, p=2, dim=-1)

        # Local coupling
        ## 循环边界 (实际上应该填充 (padding) 常数), 保护边缘 Token
        X_padded = F.pad(X_norm.unsqueeze(0),(0,0,w,w),mode = 'circular').
↪squeeze(0) # 这里为了简化, 令 batch_size = 1
        ## 滑动窗口 (unfold)
        X_local = X_padded.unfold(0,2*w+1,1).transpose(1,2)
```

```
        distances = torch.cdist(X_norm.unsqueeze(1),X_local,p=2).squeeze(1) #␣
↪p=2 表示 L2 Norm

        ## 把自己和自己的距离设为\infty
        ### 直接原地操作会导致梯度爆炸 "distances[:,w] = float('inf')"
        mask = torch.zeros_like(distances,dtype=torch.bool)
        mask[...,w] = True
        distances = distances.masked_fill(mask,float('inf'))
        # 求 top-k
        topk_dist,topk_index = torch.topk(distances, self.k, largest=False)
        topk_neighbours = X_local[torch.arange(N,device=X.device).
↪reshape(-1,1),topk_index]

        # RBF Kernel
        unnormalized_weights = torch.exp(-self.gamma * torch.square(topk_dist))

        # L1 归一化
        eps = 1e-8
        attention_weights = unnormalized_weights/(torch.
↪sum(unnormalized_weights, dim=-1,keepdim=True)+eps)

        # 保存用于调试
        self._attention_weights = attention_weights
        self._topk_dist = topk_dist

        # 加权聚合
        Y = torch.einsum('nk,nkd->nd', attention_weights, topk_neighbours)

        # 残差连接
        X_new = self.layernorm(X + Y)
        return X_new

        '''
        各 Tensor 的维度分析:
```

```
            变量名                 维度                          说明
```

变量名	维度	说明
X	(N, d)	输入，N=token 个数，d=embedding 维度
self.gamma	(1,)	可学习参数，控制距离衰减
w	标量	窗口半径
X_padded	(N+2w, d)	循环填充后的序列
X_local	(N, 2w+1, d)	每个 token 的局部窗口 embedding
X.unsqueeze(1)	(N, 1, d)	扩展维度用于 cdist
distances	(N, 2w+1)	每个 token 到窗口内邻居的距离
topk_dist	(N, k)	每个 token 到其 k 个最近邻居的距离
topk_index	(N, k)	每个 token 的 k 个最近邻居的索引
topk_neighbours	(N, k, d)	邻居的 embedding 向量
unnormalized_weights	(N, k)	未归一化的注意力权重

```
        attention_weights      (N, k)         归一化后的注意力权重                 ␣
↪
                                                                                  ␣
↪
        Y                      (N, d)         加权聚合后的邻居 embedding           ␣
↪
                                                                                  ␣
↪
        X_new                  (N, d)         LayerNorm 后的残差输出               ␣
↪

    '''
```

1.4 尝试运行测试

by claude code

1.4.1 类定义

[39]:
```
# ==========================================
# 支持批量训练的 Krause 注意力 (带投影层)
# ==========================================
class KrauseAttentionBatch(nn.Module):
    """Krause 注意力: 带 QKV 投影, 参数量与标准注意力相当"""
    def __init__(self, d: int, k: int, window: int, gamma: float = 1.0):
        super().__init__()
        self.k = k
        self.window = window
        self.d = d

        # 投影层 (与标准注意力相同)
        self.q_proj = nn.Linear(d, d)
        self.k_proj = nn.Linear(d, d)
        self.v_proj = nn.Linear(d, d)
        self.out_proj = nn.Linear(d, d)
```

```python
        # Krause 特有参数
        self.gamma = nn.Parameter(torch.tensor([gamma], dtype=torch.float32))
        self.layernorm = nn.LayerNorm(d)

    def forward(self, X: torch.Tensor) -> torch.Tensor:
        B, N, d = X.shape
        w = self.window

        # QKV 投影
        Q = self.q_proj(X)
        K = self.k_proj(X)
        V = self.v_proj(X)

        # L2 归一化 Q 和 K
        Q_norm = F.normalize(Q, p=2, dim=-1)
        K_norm = F.normalize(K, p=2, dim=-1)

        # 循环填充 K 和 V
        K_padded = F.pad(K_norm, (0, 0, w, w), mode='circular')
        V_padded = F.pad(V, (0, 0, w, w), mode='circular')

        # 构建局部窗口
        K_local = K_padded.unfold(1, 2*w + 1, 1).permute(0, 1, 3, 2)  # (B, N,␣
↪2w+1, d)
        V_local = V_padded.unfold(1, 2*w + 1, 1).permute(0, 1, 3, 2)

        # 计算距离
        Q_expanded = Q_norm.unsqueeze(2)  # (B, N, 1, d)
        distances = torch.norm(Q_expanded - K_local, p=2, dim=-1)  # (B, N,␣
↪2w+1)

        # 屏蔽自身
        mask = torch.zeros_like(distances, dtype=torch.bool)
        mask[..., w] = True
        distances = distances.masked_fill(mask, float('inf'))
```

```python
        # Top-k 最近邻
        topk_dist, topk_idx = torch.topk(distances, self.k, largest=False)

        # 收集邻居
        batch_idx = torch.arange(B, device=X.device)[:, None, None]
        token_idx = torch.arange(N, device=X.device)[None, :, None]
        topk_V = V_local[batch_idx, token_idx.expand(B, N, self.k), topk_idx]

        # RBF 权重
        weights = torch.exp(-self.gamma * topk_dist.square())
        attention = weights / (weights.sum(dim=-1, keepdim=True) + 1e-8)

        # 加权聚合
        Y = torch.einsum('bnk,bnkd->bnd', attention, topk_V)

        # 输出投影 + 残差
        Y = self.out_proj(Y)
        return self.layernorm(X + Y)

# ==========================================
# 标准全局注意力 (带投影层)
# ==========================================
class GlobalAttention(nn.Module):
    """ 标准缩放点积注意力: 带 QKV 投影"""
    def __init__(self, d: int):
        super().__init__()
        self.d = d
        self.scale = d ** 0.5

        # 投影层
        self.q_proj = nn.Linear(d, d)
        self.k_proj = nn.Linear(d, d)
        self.v_proj = nn.Linear(d, d)
        self.out_proj = nn.Linear(d, d)

        self.layernorm = nn.LayerNorm(d)
```

```
    def forward(self, X: torch.Tensor) -> torch.Tensor:
        # QKV 投影
        Q = self.q_proj(X)
        K = self.k_proj(X)
        V = self.v_proj(X)

        # 全局注意力
        scores = torch.bmm(Q, K.transpose(1, 2)) / self.scale
        attn = torch.softmax(scores, dim=-1)
        Y = torch.bmm(attn, V)

        # 输出投影 + 残差
        Y = self.out_proj(Y)
        return self.layernorm(X + Y)
```

1.4.2 实验

[40]:
```
import numpy as np
import matplotlib.pyplot as plt
import time

# ==========================================
# 实验 1: 序列长度 vs 推理速度
# ==========================================

print("=" * 60)
print("实验 1: 序列长度 vs 推理速度")
print("=" * 60)

d, batch_size, k, window = 64, 8, 5, 10
seq_lengths = [64, 128, 256, 512, 1024, 2048]

times_krause, times_global = [], []
for seq_len in seq_lengths:
    X = torch.randn(batch_size, seq_len, d, dtype=torch.float32)
```

```
    model_k = KrauseAttentionBatch(d, k, window)
    model_g = GlobalAttention(d)

    _ = model_k(X)
    _ = model_g(X)

    start = time.time()
    for _ in range(10):
        _ = model_k(X)
    times_krause.append((time.time() - start) / 10 * 1000)

    start = time.time()
    for _ in range(10):
        _ = model_g(X)
    times_global.append((time.time() - start) / 10 * 1000)

    print(f"N={seq_len:4d}: Krause={times_krause[-1]:6.2f}ms,␣
↪Global={times_global[-1]:6.2f}ms, speedup={times_global[-1]/times_krause[-1]:
↪.2f}x")

# ==========================================
# 实验 2: 完整模型训练对比
# ==========================================

print("\n" + "=" * 60)
print("实验 2: 完整模型训练对比")
print("=" * 60)

class TransformerBlock(nn.Module):
    """ 完整的 Transformer Block"""
    def __init__(self, d, attention_type='krause', k=5, window=10, ffn_ratio=4):
        super().__init__()
        if attention_type == 'krause':
            self.attn = KrauseAttentionBatch(d, k, window)
        else:
            self.attn = GlobalAttention(d)
```

```python
        self.ffn = nn.Sequential(
            nn.Linear(d, d * ffn_ratio),
            nn.GELU(),
            nn.Linear(d * ffn_ratio, d)
        )
        self.norm1 = nn.LayerNorm(d)
        self.norm2 = nn.LayerNorm(d)

    def forward(self, x):
        x = x + self.attn(self.norm1(x))
        x = x + self.ffn(self.norm2(x))
        return x

class CompleteModel(nn.Module):
    """ 完整的序列模型"""
    def __init__(self, d, n_layers, attention_type='krause', k=5, window=10):
        super().__init__()
        self.embed = nn.Linear(d, d)
        self.blocks = nn.ModuleList([
            TransformerBlock(d, attention_type, k, window)
            for _ in range(n_layers)
        ])
        self.head = nn.Linear(d, d)

    def forward(self, x):
        x = self.embed(x)
        for block in self.blocks:
            x = block(x)
        return self.head(x)

def generate_smooth_dataset(n_samples, seq_len, d, neighbor_radius=3, seed=42):
    """ 生成局部平滑数据集"""
    torch.manual_seed(seed)
    X_all = torch.randn(n_samples, seq_len, d, dtype=torch.float32)
    Y_all = torch.zeros_like(X_all)
```

```
    for i in range(n_samples):
        X_padded = F.pad(X_all[i:i+1], (0, 0, neighbor_radius,␣
↪neighbor_radius), mode='circular')
        for j in range(2 * neighbor_radius + 1):
            Y_all[i] += X_padded[0, j:j+seq_len, :]
        Y_all[i] /= (2 * neighbor_radius + 1)

    return X_all, Y_all

# 训练参数
seq_len, d, n_samples, batch_size = 256, 32, 500, 32
k, window, n_layers = 5, 10, 2
n_epochs = 100

# 固定数据集
X_dataset, Y_dataset = generate_smooth_dataset(n_samples, seq_len, d)
print(f"Dataset: {n_samples} samples, seq_len={seq_len}, d={d},␣
↪layers={n_layers}")

# 创建两个模型
torch.manual_seed(42)
model_krause = CompleteModel(d, n_layers, 'krause', k, window)
torch.manual_seed(42)
model_global = CompleteModel(d, n_layers, 'global')

params_k = sum(p.numel() for p in model_krause.parameters())
params_g = sum(p.numel() for p in model_global.parameters())
print(f"Parameters: Krause={params_k:,}, Global={params_g:,}")

optimizer_k = torch.optim.AdamW(model_krause.parameters(), lr=1e-3,␣
↪weight_decay=0.01)
optimizer_g = torch.optim.AdamW(model_global.parameters(), lr=1e-3,␣
↪weight_decay=0.01)

# 训练
losses_k, losses_g = [], []
```

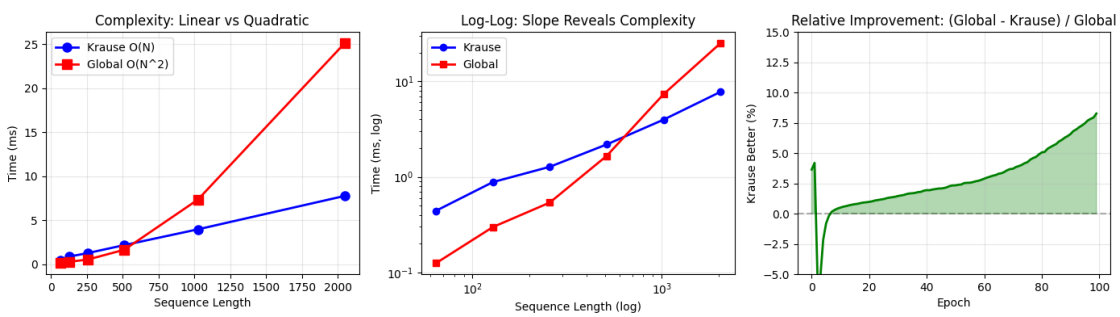
```
indices = np.arange(n_samples)

for epoch in range(n_epochs):
    np.random.seed(epoch)
    np.random.shuffle(indices)

    epoch_loss_k, epoch_loss_g = 0, 0
    n_batches = 0

    for i in range(0, n_samples, batch_size):
        batch_idx = indices[i:i+batch_size]
        X = X_dataset[batch_idx]
        Y = Y_dataset[batch_idx]

        optimizer_k.zero_grad()
        loss_k = F.mse_loss(model_krause(X), Y)
        loss_k.backward()
        torch.nn.utils.clip_grad_norm_(model_krause.parameters(), 1.0)
        optimizer_k.step()
        epoch_loss_k += loss_k.item()

        optimizer_g.zero_grad()
        loss_g = F.mse_loss(model_global(X), Y)
        loss_g.backward()
        torch.nn.utils.clip_grad_norm_(model_global.parameters(), 1.0)
        optimizer_g.step()
        epoch_loss_g += loss_g.item()

        n_batches += 1

    losses_k.append(epoch_loss_k / n_batches)
    losses_g.append(epoch_loss_g / n_batches)

    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}: Krause={losses_k[-1]:.4f},␣
↪Global={losses_g[-1]:.4f}")
```

```python
print(f"\nFinal: Krause={losses_k[-1]:.6f}, Global={losses_g[-1]:.6f}")

# 计算相对差异
relative_diff = [(g - k) / g * 100 for k, g in zip(losses_k, losses_g)]

# ==========================================
# 可视化 (改进版)
# ==========================================
fig, axes = plt.subplots(1, 3, figsize=(14, 4))

# 图 1: 序列长度 vs 时间
axes[0].plot(seq_lengths, times_krause, 'b-o', label='Krause O(N)',␣
 ↪linewidth=2, markersize=8)
axes[0].plot(seq_lengths, times_global, 'r-s', label='Global O(N^2)',␣
 ↪linewidth=2, markersize=8)
axes[0].set_xlabel('Sequence Length')
axes[0].set_ylabel('Time (ms)')
axes[0].set_title('Complexity: Linear vs Quadratic')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# 图 2: 对数坐标
axes[1].loglog(seq_lengths, times_krause, 'b-o', label='Krause', linewidth=2)
axes[1].loglog(seq_lengths, times_global, 'r-s', label='Global', linewidth=2)
axes[1].set_xlabel('Sequence Length (log)')
axes[1].set_ylabel('Time (ms, log)')
axes[1].set_title('Log-Log: Slope Reveals Complexity')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

# 图 3: 相对差异 (Krause 相比 Global 好多少 %)
axes[2].plot(relative_diff, 'g-', linewidth=2)
axes[2].axhline(y=0, color='k', linestyle='--', alpha=0.3)
axes[2].fill_between(range(len(relative_diff)), relative_diff, 0,
```

```
                        where=[d > 0 for d in relative_diff], alpha=0.3,␣
 ↪color='green')
axes[2].set_xlabel('Epoch')
axes[2].set_ylabel('Krause Better (%)')
axes[2].set_title('Relative Improvement: (Global - Krause) / Global')
axes[2].grid(True, alpha=0.3)
axes[2].set_ylim(-5, 15)  # 设置合适的范围

plt.tight_layout()
plt.show()

# ==========================================
# 实验总结
# ==========================================
print("\n" + "=" * 60)
print("实验总结")
print("=" * 60)
print(f"\n 复杂度优势:")
print(f"   N={seq_lengths[-1]}: Krause 快 {times_global[-1]/times_krause[-1]:.
 ↪1f}x")

print(f"\n 训练效果对比:")
print(f"   初始损失: Krause={losses_k[0]:.4f}, Global={losses_g[0]:.4f}")
print(f"   最终损失: Krause={losses_k[-1]:.6f}, Global={losses_g[-1]:.6f}")
print(f"   最终优势: Krause 优于 Global {relative_diff[-1]:.1f}%")

print("\n 核心结论:")
print("   1. 长序列场景: Krause 计算效率更高 (O(N) vs O(N²))")
print("   2. 局部任务: Krause 收敛更快、效果更好")
print("   3. 完整模型中 Krause 注意力层可即插即用")
```

```
============================================================
实验 1: 序列长度 vs 推理速度
============================================================
N=  64: Krause=  0.44ms, Global=  0.12ms, speedup=0.28x
N= 128: Krause=  0.88ms, Global=  0.30ms, speedup=0.34x
N= 256: Krause=  1.27ms, Global=  0.54ms, speedup=0.42x
```

```
N= 512: Krause=  2.18ms, Global=  1.66ms, speedup=0.76x
N=1024: Krause=  3.97ms, Global=  7.34ms, speedup=1.85x
N=2048: Krause=  7.76ms, Global= 25.10ms, speedup=3.23x

============================================================
实验 2: 完整模型训练对比
============================================================
Dataset: 500 samples, seq_len=256, d=32, layers=2
Parameters: Krause=27,650, Global=27,648
Epoch 20: Krause=0.1274, Global=0.1286
Epoch 40: Krause=0.1239, Global=0.1263
Epoch 60: Krause=0.1215, Global=0.1250
Epoch 80: Krause=0.1180, Global=0.1240
Epoch 100: Krause=0.1132, Global=0.1234

Final: Krause=0.113185, Global=0.123394
```




```
============================================================
实验总结
============================================================

  复杂度优势:
    N=2048: Krause 快 3.2x

  训练效果对比:
    初始损失: Krause=1.5354, Global=1.5935
```

最终损失: Krause=0.113185, Global=0.123394

最终优势: Krause 优于 Global 8.3%

核心结论:

1. 长序列场景: Krause 计算效率更高 ($O(N)$ vs $O(N^2)$)
2. 局部任务: Krause 收敛更快、效果更好
3. 完整模型中 Krause 注意力层可即插即用